Comparing Simulation Models of 2D Diffusion-Limited Aggregation

Jackson Bamrick

5/5/26

Abstract

This paper presents a simulation-based approach to studying diffusion-limited aggregation (DLA) using Monte-Carlo methods implemented through NumPy's random package. I utilized and compared three different simulation models: Inward Growth, Outward Growth, and Unrestricted Outward Growth. By comparing the simulation runtimes and resulting fractal geometries, I was able to demonstrate that a Unrestricted Outward Growth model optimized with a 'kill radius' provided the best computational efficiencies without compromising the realism of the resulting formations of DLA-based patterns. Ultimately, this work indicates that relatively simple random walk logic, when combined with certain constraints, can reliably produce the fractal patterns that appear in natural processes where DLA is present.

Introduction

Diffusion-Limited Aggregation (DLA) is a stochastic process in which particles undergoing Brownian motion move randomly until they aggregate and form fractal structures. DLA is a fundamental model for understanding how microscopic random processes can create complex macroscopic patterns. There are many real-world examples of DLA processes and their well-known resulting patterns (figure 1); these natural processes include frost formation, electrolytic deposition, and much more.

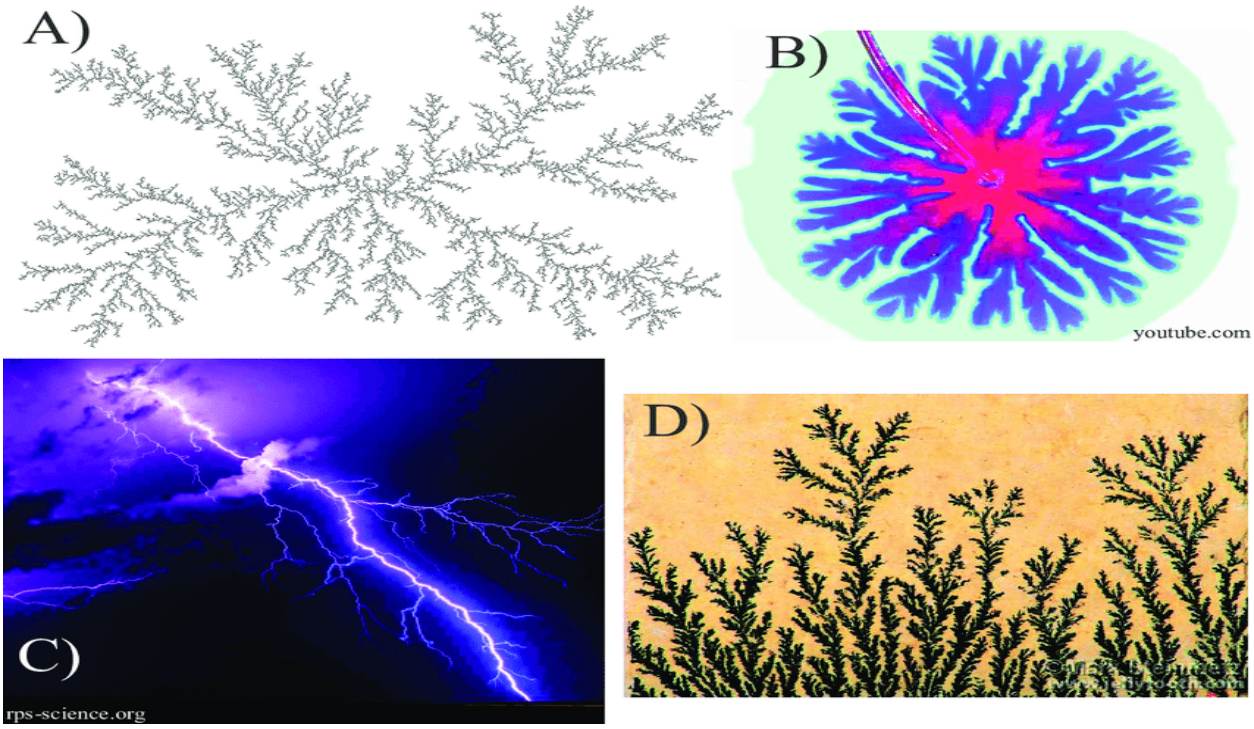

Figure 1: Different dendritic structures produced by DLA processes.
Image source: [2]

The motivation for this project stems from Exercise 10.14 (Figure 2) [1] from *Computational Physics*, written by Mark Newman.

Exercise 10.14: Diffusion-limited aggregation

This exercise builds upon Exercise 10.3 on page 461. If you have not done that exercise you should do it before doing this one.

In this exercise you will develop a computer program to reproduce one of the most famous models in computational physics, *diffusion-limited aggregation*, or DLA for short. There are various versions of DLA, but the one we will study is as follows. You take a square grid with a single particle in the middle. The particle performs a random walk from square to square on the grid until it reaches a point on the edge of the system, at which point it "sticks" to the edge, becoming anchored there and immovable:

Figure 2: Exercise 10.14

Beyond the motivations from Exercise 10.14 [1], the goal of this project was to create a simulation program capable of recreating the emergent geometries associated with natural DLA processes in the most computationally efficient way possible. Monte Carlo simulations were determined to be a useful tool for achieving this goal for multiple reasons. First off, DLA is a stochastic process, meaning that particle's motions are random and cannot be predicted with simple formulas. Additionally, thousands of particles and millions of steps must be simulated for any patterns to emerge, creating a large scale requirement. Finally, a simple 'random walk' logic can be used to accurately represent the Brownian motion that is largely responsible for DLA.

With all of this in mind, I created a program that simulates DLA as described in Exercise 10.14 [1] using an Inward Growth model, parts A&B, an Outward Growth model, part C, as well as an Unrestricted Outward Growth model that serves as an extension.

Physical Theory and Model

This simulation is guided by two core principles, Brownian Motion and Limited Aggregation. Brownian Motion is the concept that is responsible for particles in DLA processes moving in random motion; this is typically caused by outside disturbances to the system from its surroundings. In the 2D grid that I used for my simulations, Brownian Motion is implemented by a particle on a empty grid at location (*x, y*) having an equal probability of moving into any of the adjacent spots:

$$P(\Delta x, \Delta y) = 0.25 \text{ , for } (\pm 1,0) \text{ and } (0, \pm 1). \quad (1)$$

Limited Aggregation is the concept that allows the emergent geometries of DLA processes to form; it occurs when a wandering particle bumps into an anchored seed and becomes permanently stuck upon contact. In my 2D simulation, this sticking condition is implanted through empty grid sites having a value of zero and anchored seeds having a non-zero value. At each step of the simulation, the particle's four neighbors checked for vacancy to determine if the particle should stick or keep wandering.

Simulation Program

The program was written in Python using the NumPy random package to support the random walk logic, as well as the Matplotlib and the IPython libraries for visualization. The general algorithm for my simulations was the same for all three methods and is as described below:

- Initialize: A 201x201 grid full of zeros is created with a 'seed particle' filling one grid location with a non-zero value.
- Spawn: A particle is spawned in at some designated starting location.
- Random Walk: The particle undergoes simulated Brownian Motion until a sticking condition is met, simulated using the logic shown in figures 3&4.
- Anchor: The grid location at the sticking location is updated to the current particle index to track the age of the particle.
- Repeat: The process repeats for however many particles have been designated to spawn.
- Visualize: Use the IPython and Matplotlib libraries to visualize simulations, as static images of the final result and GIFs of the entire simulation run.

```python
# Function to check if particle should stick
def is_neighbor_anchored(x, y):
    # Check boundary
    if x <= 0 or x >= L-1 or y <= 0 or y >= L-1:
        return True
    # Check neighbors
    if grid[x+1, y] > 0 or grid[x-1, y] > 0 or grid[x, y+1] > 0 or grid[x, y-1] > 0:
        return True
    return False

# Function to simulate a single particle's random walk until it sticks
def simulate_particle(particle):
    x, y = center, center
    # Check if center is already blocked
    if grid[x, y] > 0:
        return None
    while True:
        # Random walk
        move = np.random.randint(4)
        if move == 0:   y += 1 # Up
        elif move == 1: y -= 1 # Down
        elif move == 2: x -= 1 # Left
        elif move == 3: x += 1 # Right
        # Check for sticking
        if is_neighbor_anchored(x, y):
            grid[x, y] = particle
            return (x, y)
```

Figure 3: Random walk & sticking logic for Inward Growth model.

```python
# Function to check if particle should stick
def is_neighbor_anchored(x, y):
    # CParticles dont stick to boundary anymore
    if x <= 0 or x >= L-1 or y <= 0 or y >= L-1:
        return False
    # Check neighbors
    if grid[x+1, y] > 0 or grid[x-1, y] > 0 or grid[x, y+1] > 0 or grid[x, y-1] > 0:
        return True
    return False

# Function to simulate particle wandering
def simulate_particle():
    # Spawn particle at random point on circle of radius r+2 around center
    x, y = spawn(state['r_max'])
    # Kill particles that wander too far from center, or hit grid edge
    r_kill = max(2.0 * state['r_max'], 5.0)
    # Wandering loop
    while True:
        move = np.random.randint(4)
        if move == 0: y += 1
        elif move == 1: y -= 1
        elif move == 2: x -= 1
        elif move == 3: x += 1
        # Check for sticking or killing
        distance = np.sqrt((x - center)**2 + (y - center)**2)
        if distance > r_kill or x <= 0 or x >= L-1 or y <= 0 or y >= L-1:
            return False
        # Check for sticking
        if is_neighbor_anchored(x, y):
            grid[x, y] = state['particle']
            state['particle'] += 1
            if distance > state['r_max']:
                state['r_max'] = distance
            return True
```

Figure 4: Random walk & sticking logic for both Outward Growth models.

Moving onto the different simulation models I used, the following are descriptions of each model along with their key features and constraints.

Inward Growth Model

In this model, built from Exercise 10.14, parts A&B [1], particles are spawned at the center of the grid, length L:

$$(X_{\text{spawn}}, Y_{\text{spawn}}) = (L/2, L/2)$$

Particles undergo the random walk process shown in Figure 4 until they collide with a structure of anchored particles or reach the boundary of the grid, as is also shown in Figure 4. This model has one main constraint: the simulation will stop once any structure reaches the center of the grid, as that would make it so no more particles can spawn. Otherwise, the simulation will run until all particles have spawned. The number of particles is determined by when creating the ani object, which is done using the FuncAnimation class;

this can be seen in Figure 5 below. Note that the HTML class is from the IPython library and is used to create a playable GIF of the simulation within the Jupyter Notebook I created within VSCode.

```
# Visual Configuration
fig, ax = plt.subplots(figsize=(6, 6))
im = ax.imshow(grid, cmap='viridis', origin='lower', interpolation='nearest')

particle_count = [1]

# Function to update viz, with particle counter
def update(frame):
    success = simulate_particle(particle_count[0])

    if success:
        im.set_data(grid)
        im.set_clim(vmin=0, vmax=particle_count[0])
        ax.set_title(f"Inward DLA Growth: {np.sum(grid > 0)} Particles Anchored")
    particle_count[0] += 1

    return [im]

# Run animation and save
# frames sets # of particles
ani = FuncAnimation(fig, update, frames=3000, interval=20, blit=True, repeat=False)
plt.close()
HTML(ani.to_jshtml())
```

Figure 5: Visual configuration and simulation control for Inward Growth Model.

Outward Growth Model

This model, built from Exercise 10.14, part C [1], is more complex than the Inward Growth Model. This complexity allows for more realistic representation of DLA patterns and for the introduction of more impactful constraints to reduce the simulation runtime. The simulation begins with one seed particle anchored at the center of the grid, accomplished using the simple logic shown in Figure 6.

```
# Simulation Configuration
L = 201  # Need L to be odd to have center pt
center = L // 2
grid = np.zeros((L, L), dtype=int)  # 0 = empty, >0 =  stuck

# Start with 1 particle in middle
grid[center, center] = 1
```

Figure 6: Seed particle initialization for Outward Growth Model.

Moving on, particles are spawned on a random point of a circle radius *r+2* ¸where r is the current farthest point a particle has traveled. This alteration was recommended by Exercise 10.14, part C [1] and implemented using the function shown in Figure 7.

```
# Function to spawn particles in radius of r+2 around center point
def spawn(r):
    # Spawn point & random angle
    r_spawn = max(r+2.0, 2.0)
    theta = np.random.uniform(0, 2*np.pi)
    # Convert to grid coords
    x = int(round((center + r_spawn * np.cos(theta))))
    y = int(round((center + r_spawn * np.sin(theta))))
    # Keep particles from spawning off the grid
    x = max(1, min(L-2, x))
    y = max(1, min(L-2, y))
    return x, y
```

Figure 7: Spawning function for Outward Growth Model

The random walk logic for this model is the same as that of the Inward Growth Model, as can be seen in Figure 4. However, Figure 4 also displays one of the new constraints: the 'kill radius', defined as:

$$r_{\text{kill}} = 2 * r_{\text{max}} \ ,$$

where r_{max} is the furthest distance that any particle has traveled; r_{max} is a part of the *state* dictionary which includes r_{max} and *particle* which is a counter for the number of particles spawned. The *particle* variable is also used for color-based age tracking, a recommendation given by Exercise 10.14, part C [1]. This age tracking is implemented in the simulation control, along with the second new constraint: the simulation will stop if any structure reaches half of the distance to the grid edge. These implementations can be seen clearly in Figure 8.

```
# Visual Configuration
fig, ax = plt.subplots(figsize=(6, 6))
im = ax.imshow(grid, cmap='plasma', origin='lower', interpolation='nearest')

# Function to update viz, with particle counter
def update(frame):
    # Stop sim if r gets farther than 1/2 of distance to boundary
    r_farthest = (L // 2) / 2.0
    if state['r_max'] < r_farthest:
        success = simulate_particle()
        if success:
            im.set_data(grid)
            im.set_clim(vmin=0, vmax=state['particle'])
            ax.set_title(f"Outward DLA Growth: {np.sum(grid>0)} particles anchored")
    return [im]

# Run animation and save
# frames sets # of particles
ani = FuncAnimation(fig, update, frames=3000, interval=20, blit=True, repeat=False)
plt.close()
HTML(ani.to_jshtml())
```

Figure 8: Visual configuration and simulation control for Outward Growth Model. Note that vmax has been set the particle variable for age-based coloring and the if statement comparing r_max and r_farthest.

Unrestricted Outward Growth Model

This model was created as a way to explore DLA behavior further; specifically looking at what structure formation would look like without the constraint of the simulation stopping once a particle makes it halfway to the grid boundary. With this motivation in mind, I made this model essentially a clone of the base Outward Growth Model, except with the second constraint removed. This change is reflected in the simulation control, shown in Figure 9.

```
# Visual Configuration
fig, ax = plt.subplots(figsize=(6, 6))
im = ax.imshow(grid, cmap='plasma', origin='lower', interpolation='nearest')

# Function to update viz, with particle counter
def update(frame):
    # Let sim run continuously until a particle successfully sticks
    success = False
    while not success:
        success = simulate_particle()

    # Once a particle sticks (success is True), update the visualization
    im.set_data(grid)
    im.set_clim(vmin=0, vmax=state['particle'])
    ax.set_title(f"Unrestricted Outward DLA Growth: {np.sum(grid>0)} particles anchored")

    return [im]

# Run animation and save
# frames sets # of particles
ani = FuncAnimation(fig, update, frames=3000, interval=20, blit=True, repeat=False)
plt.close()
HTML(ani.to_jshtml())
```

Figure 9: Visual configuration and simulation control for Unrestricted Outward Growth Model. Note the Boolean logic that allows the simulation to spawn all particles uninterrupted.

Results

The following table contains the key results from the three different simulation models.

Table 1: Key simulation results.

Method	Particles Anchored	Total Runtime	Performance Note
Inward	2,814	262 seconds	Least efficient, most particles.
Outward	756	178 seconds	Most efficient, least particles.
Unrestricted	2,505	211 seconds	Balance of efficiency and realistic DLA patterns.

Table 1 illustrates the balance of computational efficiency and simulation realism provided by the Unrestricted Outward Growth Model. The Inward Growth Model does provide an accurate picture of DLA patterns, see Figure 10, it does as the cost of nearly 4.5 minutes of runtime. The Outward Growth model on the other hand ran very quickly, under 3 minutes, but provided little benefit in terms of learning about how structures form in DLA proccesses, see Figure 11. Thankfully, the Outward Growth Model provides the best of both worlds by running for roughly 3.5 minutes and still providing an informative picture of DLA patterns, see Figure 12. This result is an example of how constraints can be added or removed in the world of simulations to maximize realism and information, while minimizing computational costs.

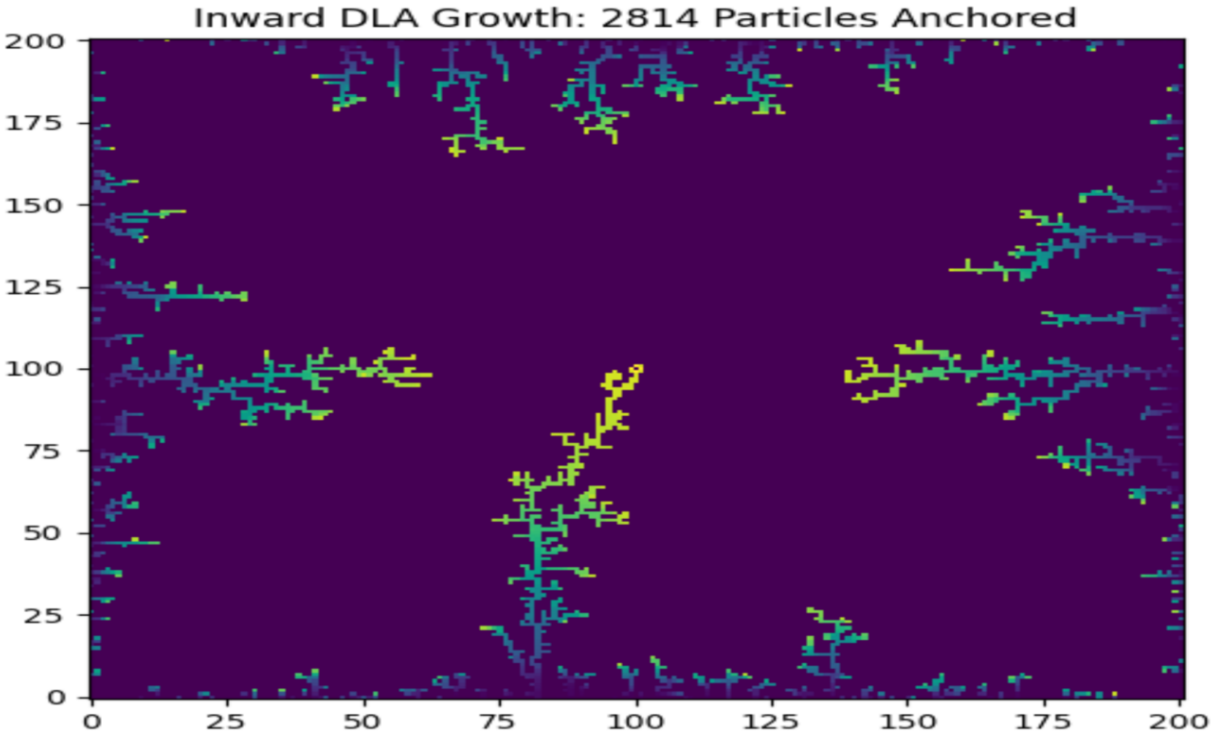

Figure 10: Inward Growth Model simulation result.

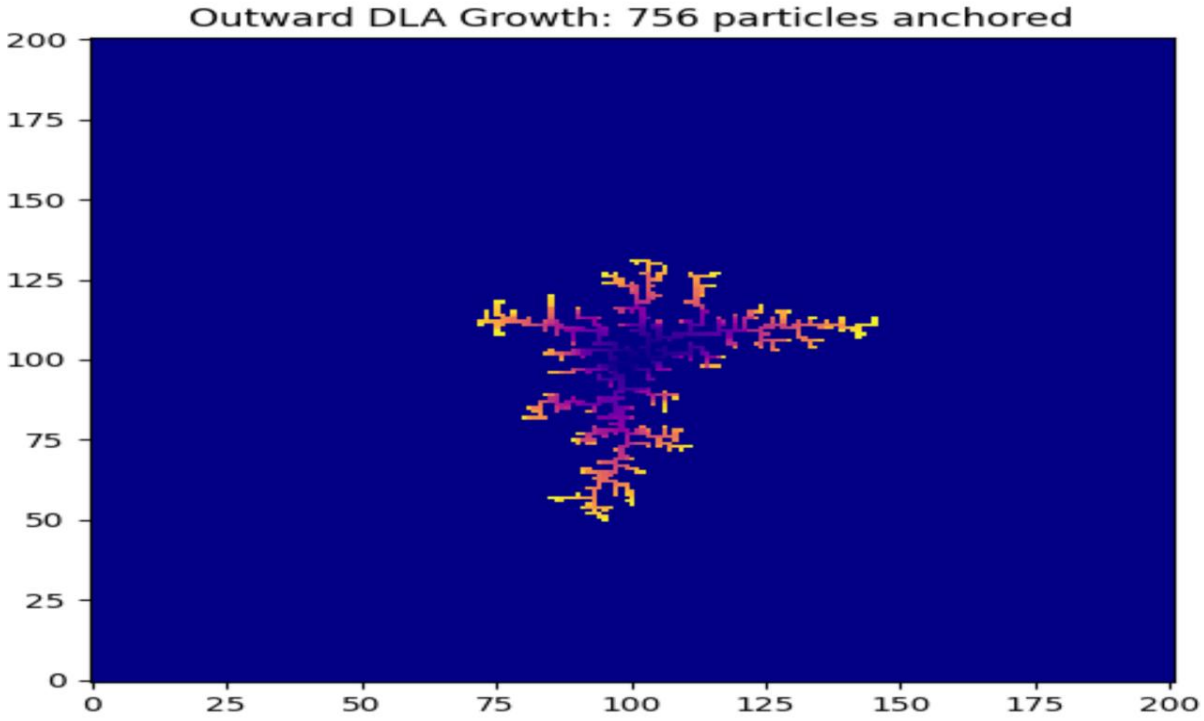

Figure 11: Outward Growth Model simulation results.

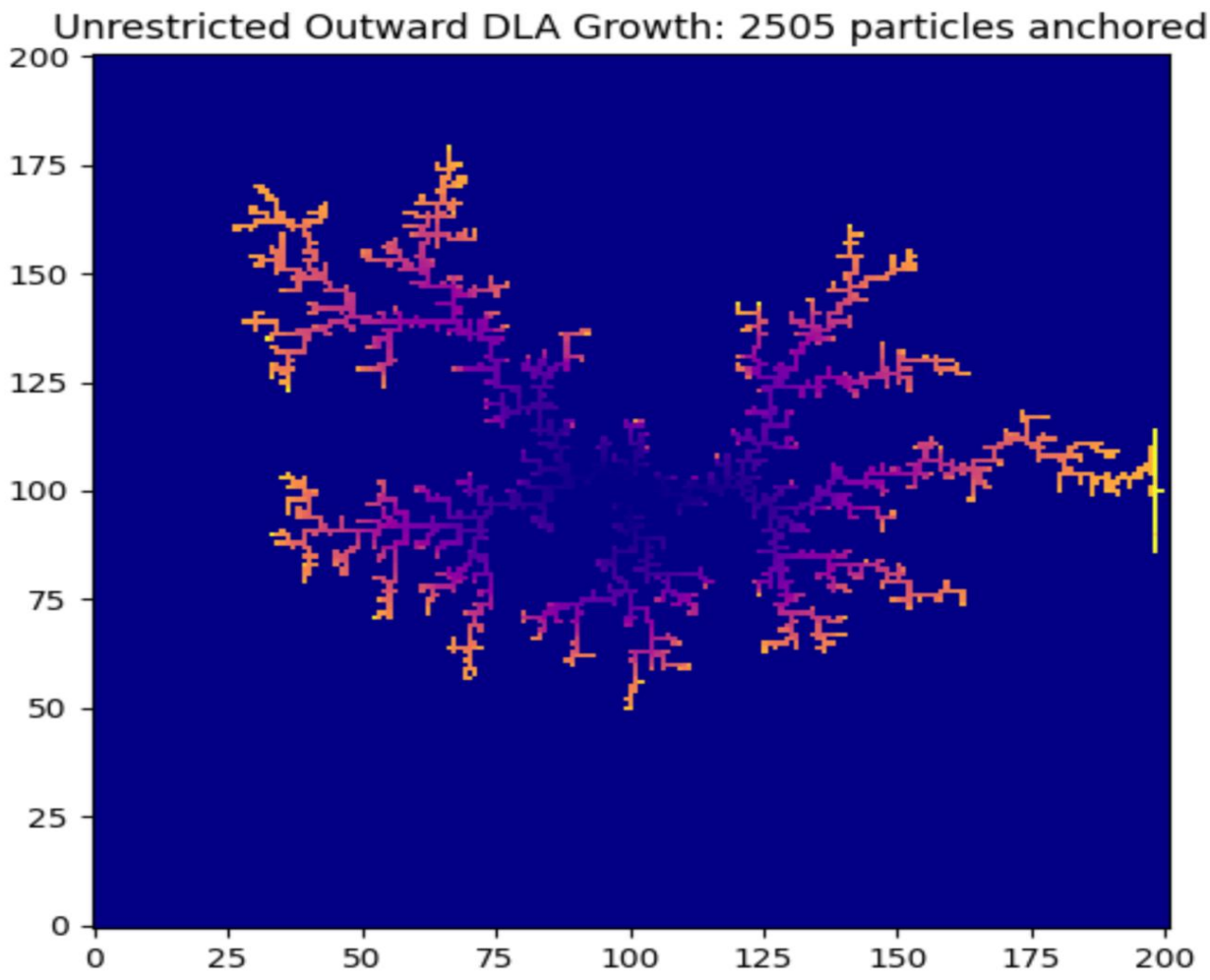

Figure 12: Unrestricted Outward Growth Model simulation results.

Concluding Remarks

In conclusion, this project successfully used three models, Inward Growth, Outward Growth, and Unrestricted Outward Growth, to simulate the formation of structures found in DLA proccesses. Results showed that the Unrestricted Outward Growth Model provided the best balance of minimizing runtime requirements without comprising any insights into DLA-based patterns. Future variations of this work might run simulations on larger grids or develop constraints to increase the efficiency of the Inward Growth Model.

References

1. Newman, Mark. (2025). *Computational Physics*. 2nd edition.
2. Flow in evolving fractures and porous media - Scientific Figure on ResearchGate. Available from: https://www.researchgate.net/figure/Dendritic-patterns-in-different-systems-A-Radial-DLA-simulation-B-Viscous-fingers-in-a_fig5_322676454 [accessed 4 May 2026]